

Pipeline topologica del diagramma elettrico

Da YOLOv11 exp11b al grafo — MVP validato su pilot_001



Obiettivo della presentazione

- riassumere il workflow completo costruito sul file di avanzamento
- mostrare i risultati ottenuti sul caso pilota pilot_001
- evidenziare limiti attuali e sviluppi successivi

Messaggio chiave

Il prototipo MVP trasforma il diagramma da semplice immagine a struttura logica interrogabile, passando per detection, terminali, fili, net e grafo finale.

Contesto, obiettivo e perimetro dell'MVP

Perché si passa dalla sola object detection alla ricostruzione topologica

Obiettivo della fase

- partire da **YOLOv11 exp11b** come base stabile di detection
- associare **identità univoche** ai componenti dello schema
- stimare i **terminali** dei componenti semplici
- estrarre i **collegamenti**, costruire le **net** e arrivare al **grafo**

Scelta progettuale iniziale

- lavorare su una singola **immagine pilota** per validare ogni blocco
- limitarsi a componenti semplici e al Mosfet solo come maschera
- rinviare OCR, testo e componenti multi-terminale complessi a una fase successiva

Pipeline logica del MVP

1. Detect components



2. Assign instances



3. Estimate terminals



4. Extract wires



5. Build nets



6. Match terminals to nets



7. Export graph

Deliverable finale del pilot: un grafo con nodi Diagram / Component / Terminal / Net e archi di connettività.

Preparazione iniziale: dataset, classi e metadati

Dai file di riepilogo classi alla definizione di `class_terminals.yaml`

File preliminari

prodotti

- **`class_summary_global.csv`**: conteggio totale delle classi nel dataset
- **`class_summary_by_split.csv`**: distribuzione per train, valid e test
- **`class_terminals.yaml`**: nome classe, tipo simbolico, uso per terminali e masking

Scopo pratico di questa

fase

- capire quali classi sono abbastanza rappresentate
- selezionare un sottoinsieme gestibile per l'MVP
- decidere quali oggetti servono per terminali e quali solo per masking

Classi selezionate per

pilot_001

- 31 — Voltage_Source
- 22 — Resistor
- 4 — Capacitor
- 26 — Terminal
- 9 — GND
- 16 — Mosfet

Regole in

`class_terminals.yaml`

- **Voltage_Source, Resistor, Capacitor, Terminal e GND** sono usati per la stima dei terminali
- **Mosfet**: `use_for_terminals = false` e `use_for_masking = true`
- questa separazione consente di semplificare l'MVP senza perdere il mascheramento dei simboli complessi

Step 01–02 | Detection dei componenti e assegnazione delle istanze

Dalla detection di YOLOv11 exp11b agli ID univoci per ogni oggetto nello schema

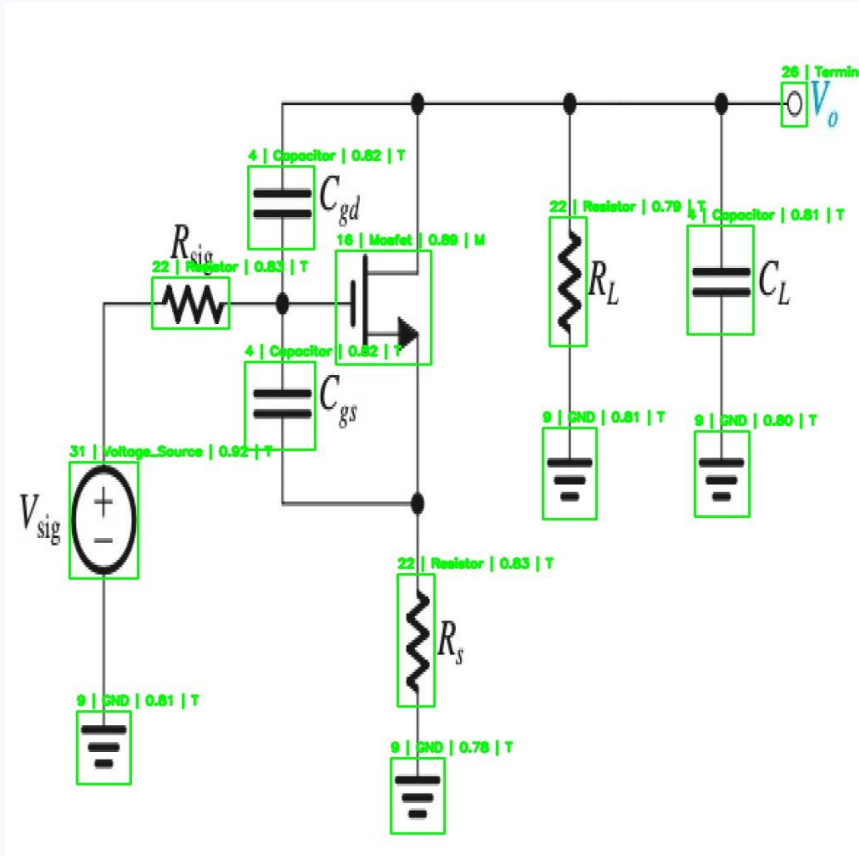


Figura 1 — pilot_001_detect.jpg | Bounding box dei componenti rilevati

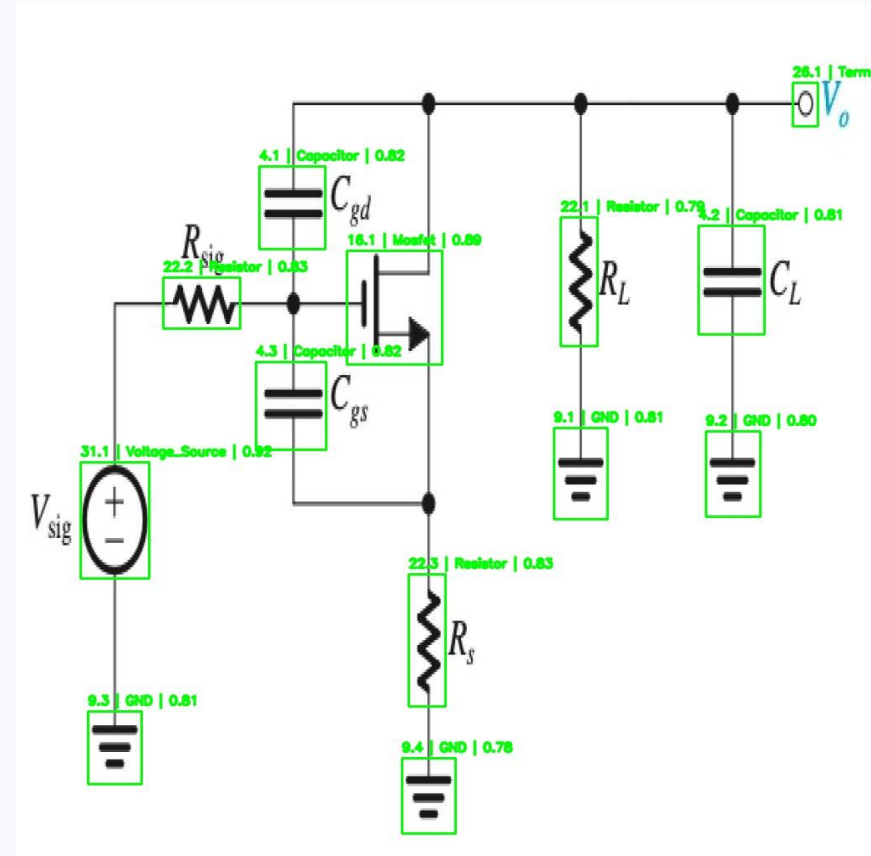


Figura 2 — pilot_001_instances.jpg | Ogni componente riceve un instance_id

Risultato chiave

Su pilot_001 la detection produce 13 componenti: 1 Terminal, 3 Capacitor, 3 Resistor, 1 Mosfet, 4 GND e 1 Voltage_Source. Lo step 02 li trasforma in istanze univoche (es. 22.1, 22.2, 4.1, 9.1).

Step 03 | Stima dei terminali

Introduzione della logica di orientazione automatica per resistor e capacitor

Problema affrontato

- la **prima versione** usava **posizioni fisse** left/right o top/bottom
- nel pilot resistor e capacitor compaiono sia orizzontali sia verticali
- serviva una regola geometrica per scegliere automaticamente l'orientazione

Soluzione adottata

- se il bbox è più alto che largo => verticale
- Voltage_Source, GND e Terminal mantengono strategia fissa (per ora)
- nel pilot vengono stimati 19 terminali complessivi

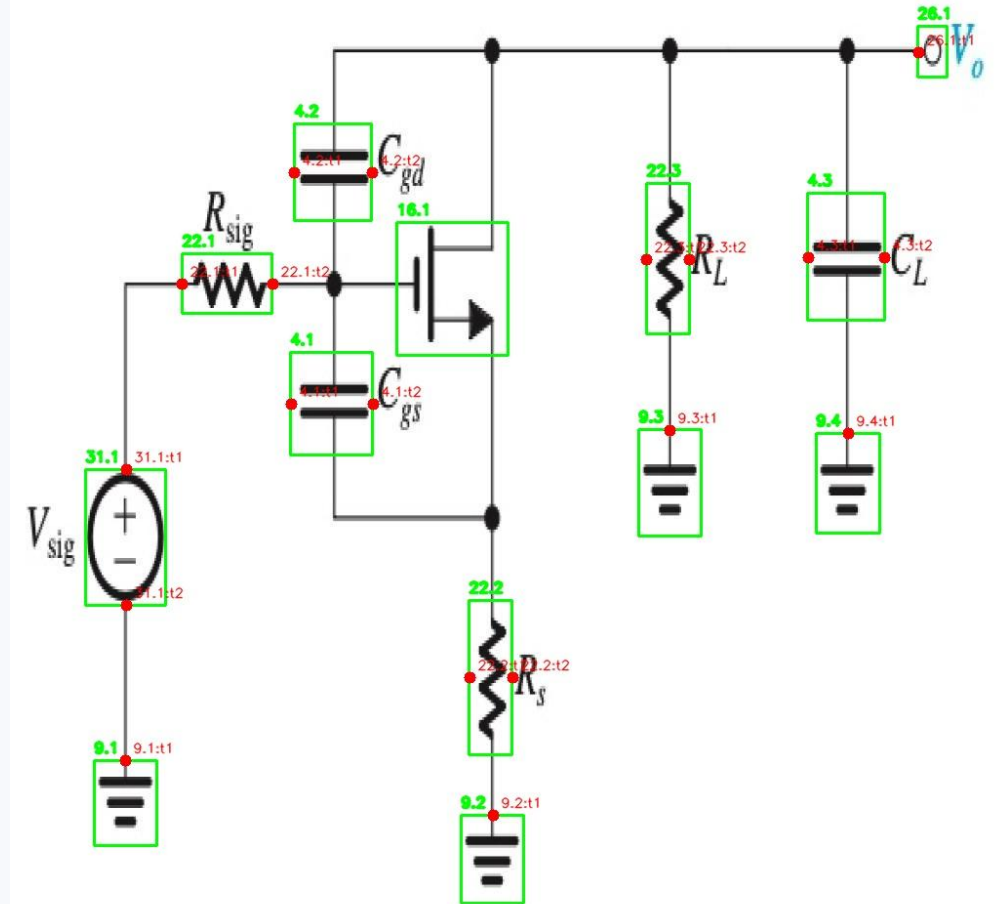


Figura 3 — pilot_001_terminals.jpg | Terminali stimati e identificati

Step 04 | Estrazione dei fili — mascheramento e preparazione

Separazione del layer dei collegamenti dal layer dei simboli

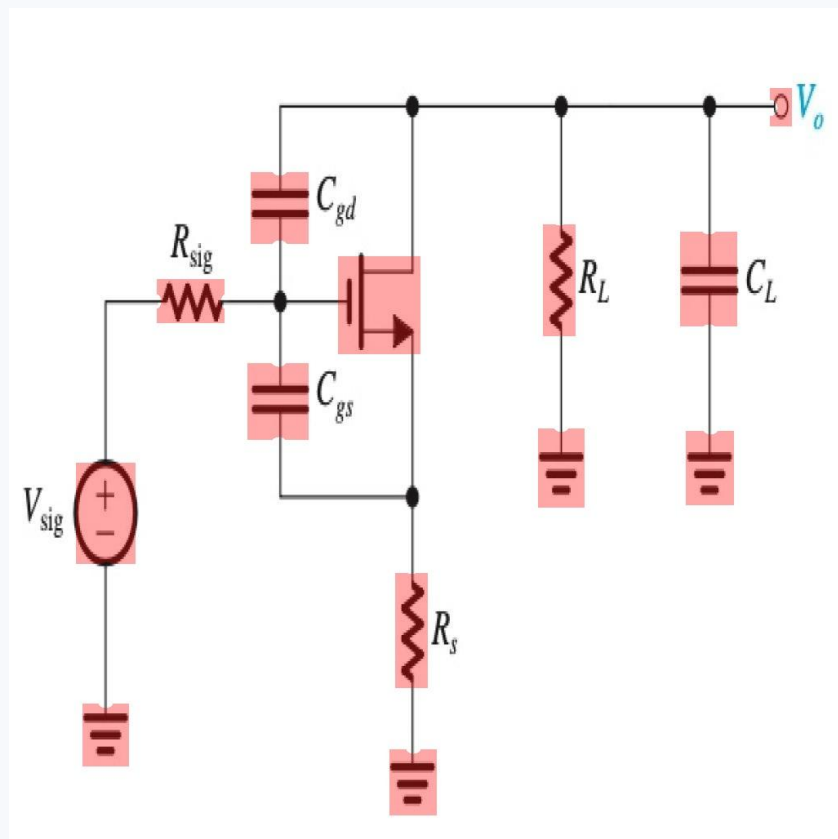


Figura 4 — pilot_001_mask_debug.jpg | Overlay della maschera

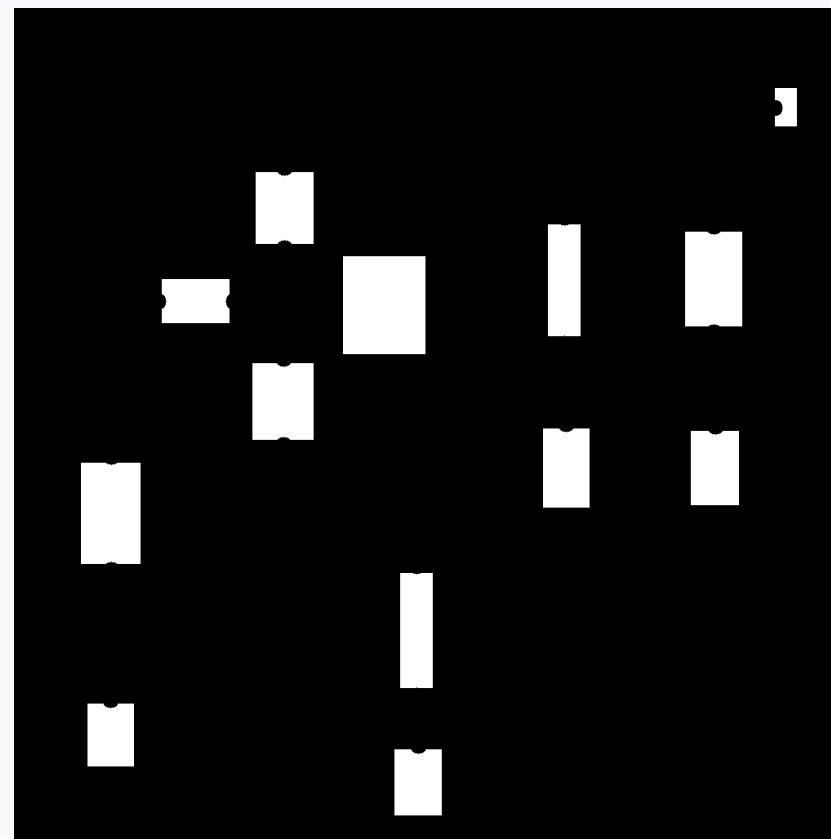


Figura 5 — pilot_001_component_mask.png | Maschera binaria pura

Parametri usati

mask_shrink_factor = 0.88 | terminal_keep_radius = 10 | closing_kernel_size = 3 | closing_iterations = 1 | small_component_filter = true (min area = 40)

Step 04 | Estrazione dei fili — pulizia, filtro e skeleton

La topologia dei collegamenti rimane leggibile, pur con testo residuo ancora presente

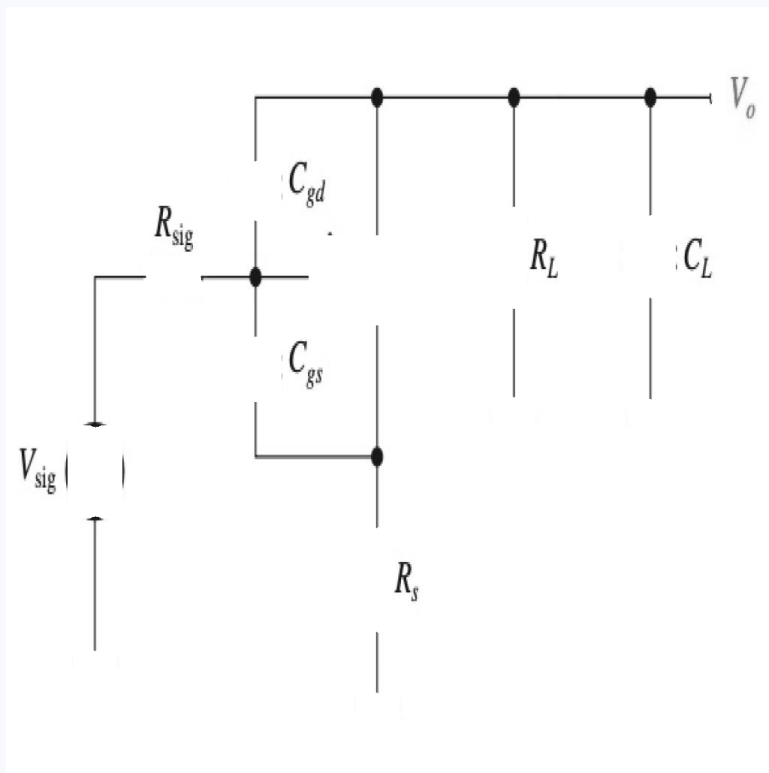


Figura 6 — pilot_001_masked_gray.png | Componenti rimossi, fili conservati

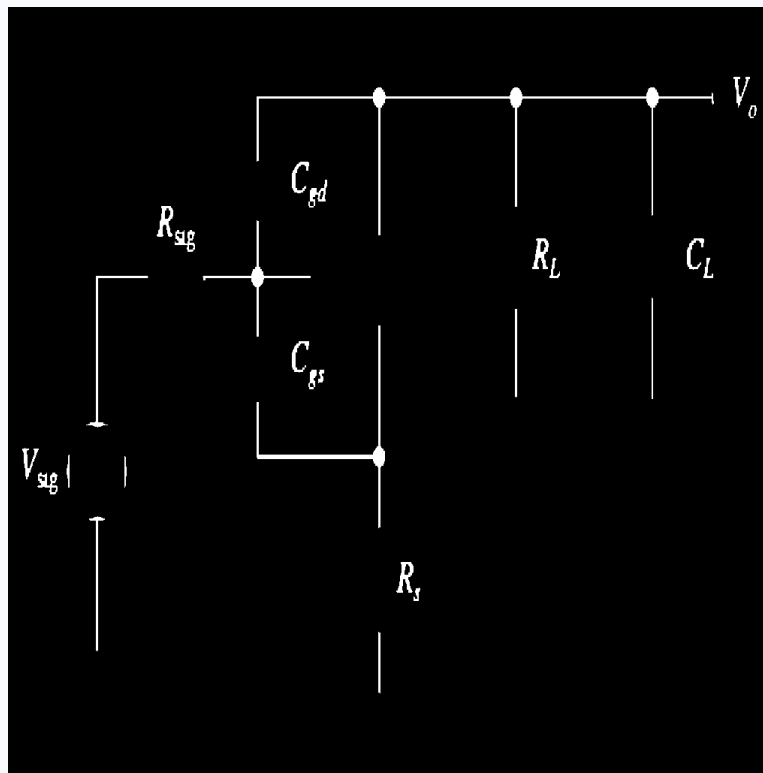


Figura 9 — pilot_001_filtered.png | Riduzione del rumore

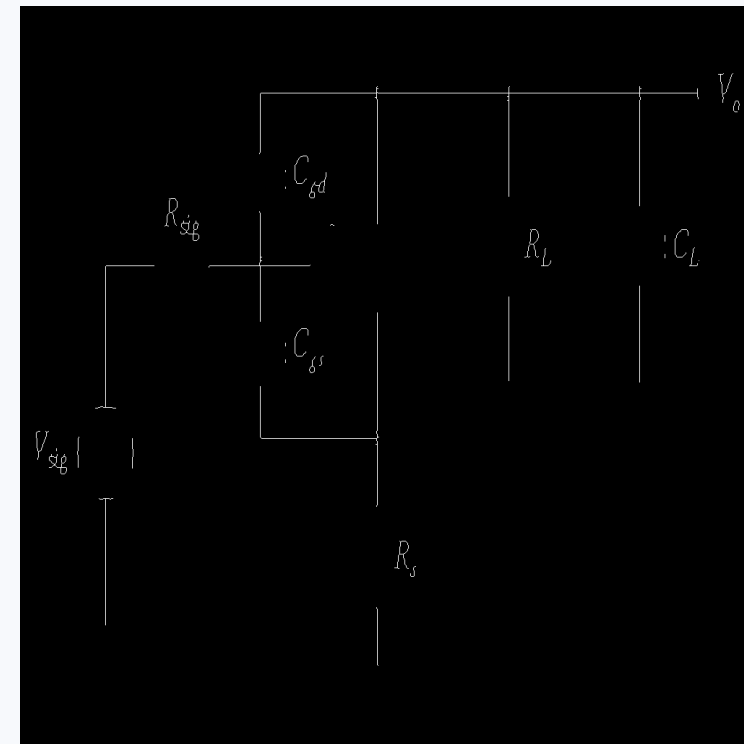


Figura 10 — pilot_001_skeleton.png | Traccia finale dei collegamenti

Step 05 | Costruzione delle net

Dallo skeleton dei fili a una prima rappresentazione esplicita delle reti elettriche

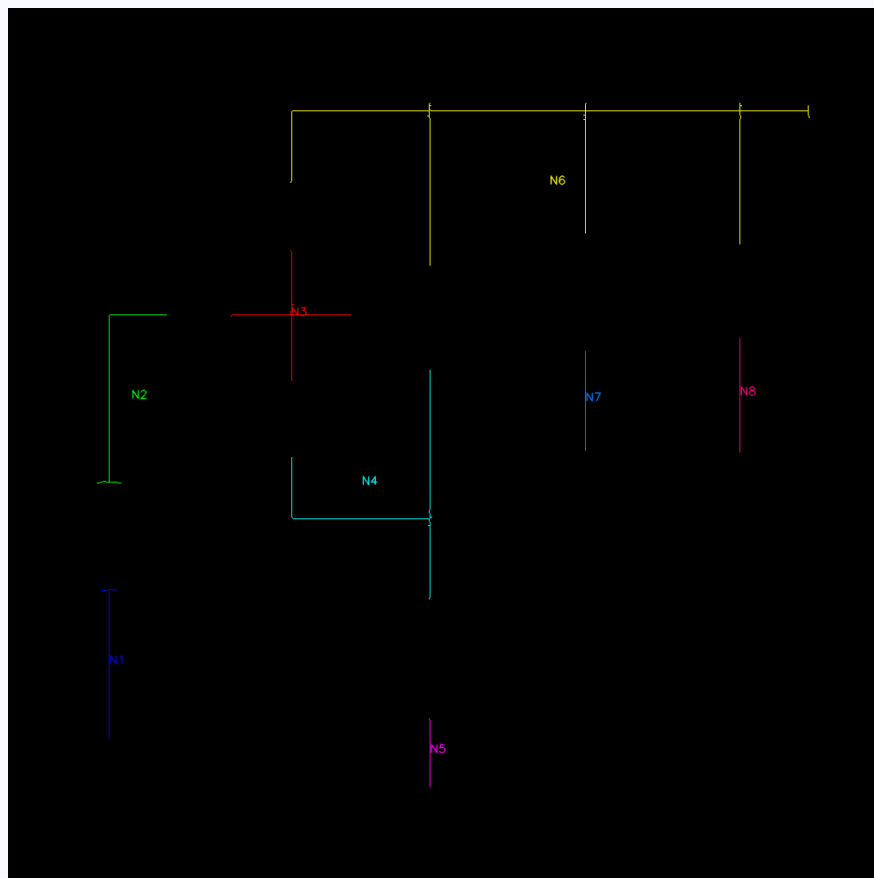


Figura 11 — pilot_001_net_map.png | Connected components mantenute come net

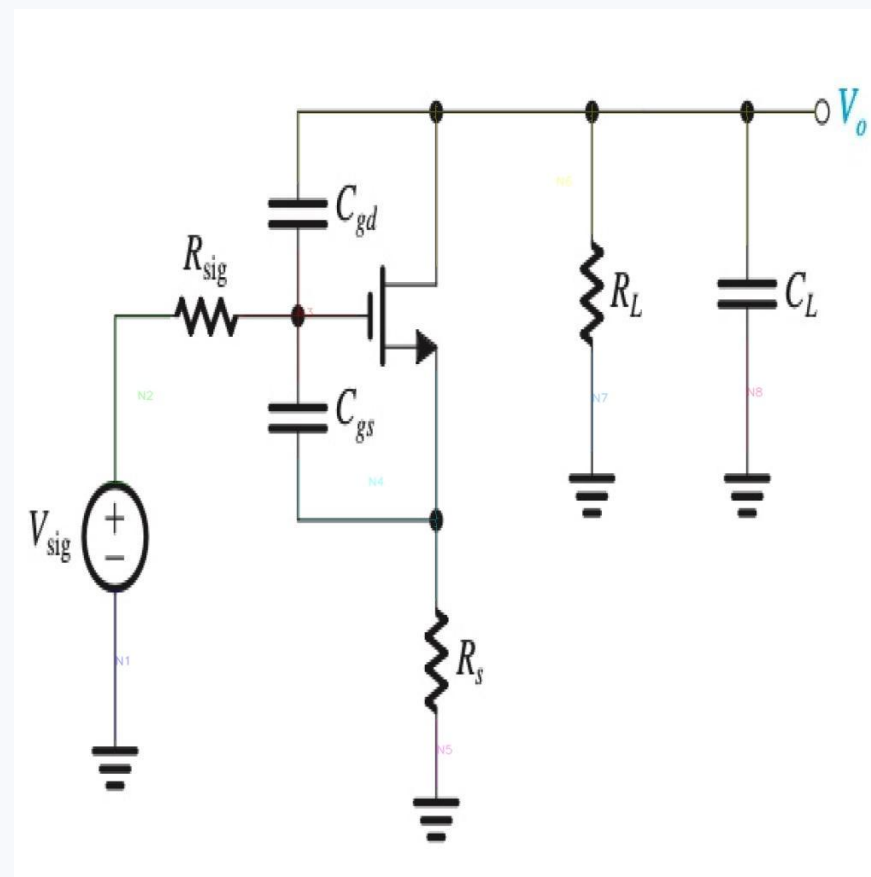


Figura 12 — pilot_001_net_overlay.jpg | Net sovrapposte all'immagine

Risultati principali

27 connected components candidate → 8 net mantenute → 19 componenti scartate. Le net N1–N8 sono coerenti con i rami principali del circuito, anche se GND non è ancora fuso semanticamente e il Mosfet spezza la continuità.

Step 06 | Matching terminale-net

Ogni terminale viene associato in modo esplicito alla net a cui appartiene

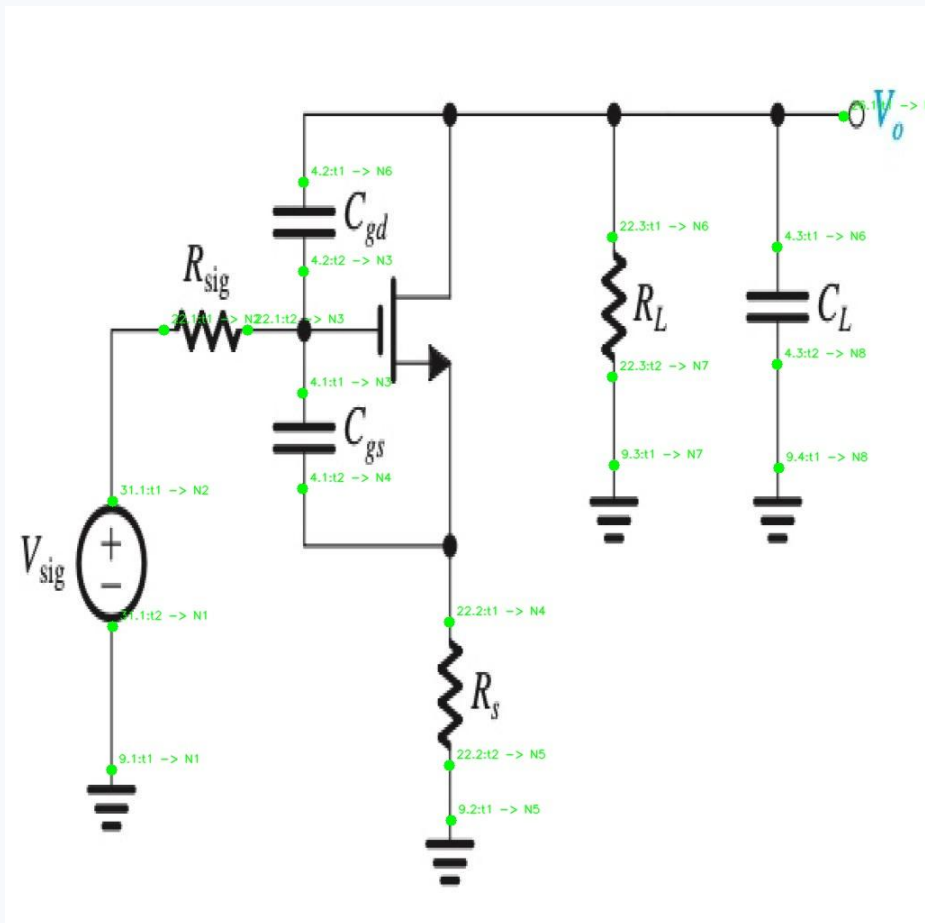


Figura 13 — pilot_001_terminal_net_matches.jpg | 19 terminali associati alle rispettive net

Numeri chiave

19 terminali stimati

19 terminali matchati

0 unmatched

Esempi di associazione

- 31.1:t1 → N2 | 31.1:t2 → N1
- 22.1:t1 → N2 | 22.1:t2 → N3
- 4.2:t1 → N6 | 4.2:t2 → N3
- 22.3:t1 → N6 | 22.3:t2 → N7

Step 07 | Esportazione del grafo

Il circuito viene trasformato in una struttura finale con nodi e archi

Schema del grafo

Diagram

pilot_001



Component

22.1 / 4.2 / 31.1 ...



Terminal

22.1:t1 / 4.2:t2 ...



Net

N1 / N2 / N3 ...

Relazioni esportate

- Diagram → HAS_COMPONENT → Component
- Diagram → HAS_NET → Net
- Component → HAS_TERMINAL → Terminal
- Terminal → CONNECTED_TO → Net

Riepilogo quantitativo del pilot

41

nodi totali

59

archi totali

13

component

19

terminal

8

net

Output finali: graph_json, nodes_csv,
edges_csv

Conclusioni e sviluppi successivi

Cosa è già stato dimostrato e quali estensioni sono più rilevanti per la tesi

Cosa dimostra il pilot

- la pipeline completa 01–07 funziona su un caso reale
- i terminali dei componenti semplici possono essere stimati in modo credibile
- dai fili estratti si arriva a net, match terminale-net e grafo finale
- il diagramma non è più solo immagine ma struttura logica interrogabile

Limiti attuali

- testo residuo ancora presente nello skeleton
- Mosfet presente solo come componente mascherato
- GND non ancora fuso semanticamente in una super-net di massa

Sviluppi successivi

- estendere la pipeline a un **piccolo batch di immagini**
- introdurre **terminali espliciti** per il **Mosfet** e **altri componenti complessi** (o tutti i 32 componenti)
- migliorare la rimozione del testo o l'uso dell'OCR in una fase più avanzata
- fondere semanticamente le masse e preparare l'esportazione verso Neo4j
- sfruttare il grafo per query e controlli strutturali sul circuito